IEEE-CIS Fraud Detection: A Machine Learning Approach

Tanveer Singh

2026-03-18

Introduction

Financial fraud poses a significant and growing threat to customers and businesses alike. In the context on digital commerce, fraud refers to the unauthorized use of payment for a financial gain. As e-commerce expands and fraudsters get smarter, detecting fraud in real time has become a challenge for financial institutions. Machine learning offers a powerful approach to this problem through classification, a type of supervised learning in which a model is trained on labeled historical data to assign new observations to categories. In the context of fraud detection, this means training a model to distinguish between legitimate and fraudulent transactions based on patterns learned from past transactions.

This project works with the IEEE-CIS Fraud Detection data set, a large scale real-world data set released through Kaggle in collaboration with Vesta Corporations, the world's leading payment service company. The data set contains anonymized transactions along with a rich set of engineered features like transaction metadata, device information, and behavioral signals. The primary goal of this project is to build and evaluate machine learning classification models capable of accurately distinguishing fraudulent transactions from legitimate ones. Given the class imbalance in fraud data (fraudulent cases represent a small minority of all transactions), we assess model performance using metric Area Under the ROC Curve (ROC AUC), which measures a model's ability to separate the two classes across all classification thresholds, and the Area Under the Precision-Recall Curve (PR AUC), which focuses on model performance among positive (fraudulent) predictions and is particularly informative when the positive class is rare.

Data Citation/Link

The data used was sourced from the IEEE-CIS Fraud Detection competition, hosted on Kaggle. The competition provided two splits of the data: a training set with labeled transactions and a test set with unlabeled transactions intended for competition submissions. Since the test set does not include the target variable, isFraud, only the training data was used for this project.

Vesta Corporation. (2019).*IEEE-CIS Fraud Detection*

Kaggle: https://www.kaggle.com/competitions/ieee-fraud-detection

Reading, Cleaning, Feature Engineering

The training data is broken into two files, transaction and identity. Start by loading the data set, then merge together on TransactionID.

```
# read in the two training data sets
identity <- read_csv("ieee-fraud-detection/train_identity.csv")
transaction <- read_csv("ieee-fraud-detection/train_transaction.csv")
```

```
merged <- transaction |>
  left_join(identity, by = "TransactionID")
```

Split the data set into our training and testing sets. We set an appropriate proportion of 80/20.

```
# sample 20% for the testing data
test_sample <- merged |>
  group_by(isFraud) |> # preserve the proportion of fraud during the split
  slice_sample(prop = 0.2) |>
  ungroup()

# use the remaining 80% for training data
train_sample <- merged |>
  anti_join(test_sample, by = "TransactionID")
```

Before any cleaning or feature engineering, lets take a look at the levels of our categorical factors and for missingness throughout the data.

```
train_sample |>
  select(where(is.character)) |> # select categorical columns
  map_int(~n_distinct(., na.rm = TRUE)) |> # unique values of each column
  enframe(name = "Variable", value = "Unique Levels") |>
  arrange(desc(`Unique Levels`)) |>
  kable()
```

Variable	Unique Levels
DeviceInfo	1700
id_33	244
id_31	130
id_30	75
R_emaildomain	60
P_emaildomain	59
ProductCD	5
card4	4
card6	4
id_34	4
M4	3
id_15	3
id_23	3
id_12	2
id_16	2
id_27	2
id_28	2
id_29	2
DeviceType	2

```
train_sample |>
  summarise(across(everything(), ~round(mean(is.na(.)) * 100, 1))) |> # percent
  pivot_longer(everything(), names_to = "Variable", values_to = "Pct Missing") |>
  arrange(desc(`Pct Missing`)) |>
```

```
  slice_head(n = 20) |> # top 20
  kable()
```

Variable	Pct Missing
id_24	99.2
id_07	99.1
id_08	99.1
id_21	99.1
id_22	99.1
id_23	99.1
id_25	99.1
id_26	99.1
id_27	99.1
dist2	93.6
D7	93.4
id_18	92.3
D13	89.5
D14	89.5
D12	89.0
id_03	88.8
id_04	88.8
D6	87.6
id_33	87.6
D8	87.3

Notice that several variables contain a large number of distinct values that would be impractical to use directly in a model. Also, quite a few variables exceed 99% missingness, leaving them effectively uninformative. This is handled next.

Before any further visualization or modeling, several cleaning and feature engineering steps need to be applied to both samples. P_emaildomain, R_emaildomain, id_30 (operating system), id_31 (browser), and DeviceInfo are consolidated into their most common groups, with all remaining values labeled as "other" or "missing". All character columns are converted to factors, with the target variable, isFraud, explicitly encoded as a two-level factor with labels "no" and "yes". Any columns exceeding a 95% threshold for missingness are dropped. Variable id_33 is removed as it contained 245 unique screen resolutions with 87.6% missingness, making it both sparse and too specific to be useful. Finally, TransactionID was removed as it is a unique row identifier with no predictive value.

```
clean_data <- function(df) {
  df |>
  mutate(
    # emails
    across(any_of(c("P_emaildomain", "R_emaildomain")),
           ~case_when(
             str_detect(., "gmail")  ~ "google",
             str_detect(., "yahoo|ymail|rocketmail") ~ "yahoo",
             str_detect(., "hotmail|outlook|msn|live") ~ "microsoft",
             str_detect(., "icloud|me.com|mac.com") ~ "apple",
             is.na(.) ~ "missing",
             TRUE ~ "other"
           )),
    # software
```

```
    across(any_of("id_30"), ~case_when(
      str_detect(., "Android") ~ "android",
      str_detect(., "iOS")     ~ "ios",
      str_detect(., "Windows") ~ "windows",
      str_detect(., "Mac")     ~ "mac",
      is.na(.) ~ "missing",
      TRUE ~ "other"
    )),
    # browser
    across(any_of("id_31"), ~case_when(
      str_detect(., "chrome") ~ "chrome",
      str_detect(., "safari") ~ "safari",
      str_detect(., "edge")   ~ "edge",
      str_detect(., "firefox")~ "firefox",
      is.na(.) ~ "missing",
      TRUE ~ "other"
    )),
    # clean device names
    across(any_of("DeviceInfo"), ~str_to_lower(str_extract(., "^[A-Za-z0-9]+"))),
    across(any_of("DeviceInfo"), ~replace_na(., "other"))
  ) |>
    # change character columns to factors
  mutate(across(where(is.character), as.factor)) |>
    # change target variable isFraud to no/yes
  mutate(across(any_of("isFraud"),
                ~factor(., levels = c(0, 1), labels = c("no", "yes")))) |>
    # drop 95% missing
  select((where(~ mean(is.na(.)) < 0.95) | any_of("isFraud")) &
           # drop id_33, TransactionID column
         -any_of(c("id_33", "TransactionID")))
}

# apply function to both datasets
train_sample <- clean_data(train_sample)
test_sample  <- clean_data(test_sample)
```

Exploratory Data Analysis (EDA)

```
productCD_summary <- train_sample |>
  group_by(ProductCD) |>
  summarize(
    Total = n(),
    Fraud_Count = sum(as.character(isFraud) == "yes", na.rm = TRUE),
    Fraud_Rate = round(mean(as.character(isFraud) == "yes", na.rm = TRUE) * 100, 2)
  ) |>
  arrange(desc(Fraud_Rate))

kable(productCD_summary,
      col.names = c("Product Code", "Total Transactions.", "Fraud Cases", "Fraud Rate (%)"),
      caption = "Fraud Risk by Product Category")
```

Table 3: Fraud Risk by Product Category

Product Code	Total Transactions.	Fraud Cases	Fraud Rate (%)
C	54845	6400	11.67
S	9330	550	5.89
H	26513	1268	4.78
R	30235	1137	3.76
W	351510	7176	2.04

Table 3 summarizes fraud rates across the five product categories in the training data. Product category C exhibits the highest fraud rate at 11.71%, nearly three times that of the next highest category, Notably, category W accounts for the vast majority of transactions (351,935), yet has the lowest fraud rate, while category C, despite having far fewer transactions (54,655), carries the greatest fraud risk. These differences suggest that product category is a potentially informative predictor for the classification models.

```
email_summary <- train_sample |>
  group_by(P_emaildomain) |>
  summarize(
    Total = n(),
    Fraud_Rate = round(mean(isFraud == "yes", na.rm = TRUE) * 100, 2)
  ) |>
  filter(Total > 10) |>
  arrange(desc(Fraud_Rate)) |>
  head(15)

kable(email_summary,
              caption = "Top High-Risk Email Domains (Min. 10 Transactions)")
```

Table 4: Top High-Risk Email Domains (Min. 10 Transactions)

P_emaildomain	Total	Fraud_Rate
microsoft	47603	5.35
google	182898	4.34
missing	75553	2.98
apple	6608	2.89
other	74873	2.30
yahoo	84898	2.22

Table 4 displays fraud rates by consolidated purchaser email domain group. Microsoft-affiliated email addresses (hotmail, outlook, msn, live) exhibit the highest fraud rate at 5.38%. Transactions where the email domain is missing have a fraud rate of 2.94%, which is notably higher than yahoo at 2.25%, suggesting that the absence of an email domain may itself be a signal of fraudulent activity.

```
dist_data <- train_sample |>
  filter(!is.na(dist1))

plot(density(log(dist_data$dist1[dist_data$isFraud == "no"] + 1)),
     main = "Figure 1: Log-Distance (dist1): Fraud vs Legit",
     xlab = "Log(Distance + 1)",
     col = "blue", lwd = 2)
```

```r
lines(density(log(dist_data$dist1[dist_data$isFraud == "yes"] + 1)),
      col = "red", lwd = 2)

legend("topright", legend = c("Legit", "Fraud"), col = c("blue", "red"), lwd = 2)
```

Figure 1: Log−Distance (dist1): Fraud vs Legit

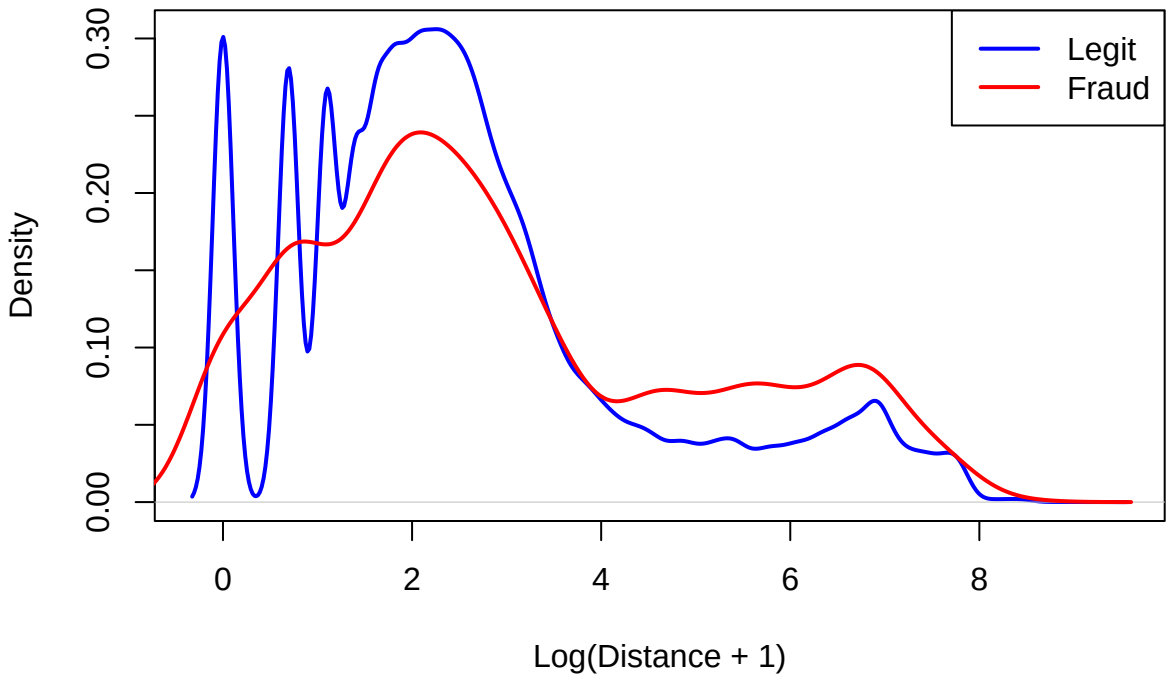


Figure 1 presents the density distributions of log-transformed dist1, representing the distance between the purchaser and the billing address, separated by fraud status. Legitimate transactions show a sharp multi-modal distribution concentrated at lower distances, with a dominant peak near a log-distance of 2. This pattern indicates that most legitimate purchases occur at relatively short distances. In contrast, fraudulent transactions exhibit a smoother and more dispersed distribution, with higher density at greater distances, particularly within the log-distance range of 4 to 8. These findings indicate that fraudulent transactions are more likely to involve greater geographic separation between the purchaser and billing address, aligning with the fraud context of this dataset, where stolen credentials are frequently used remotely.

```r
train_sample <- train_sample |>
  mutate(hour = (TransactionDT / 3600) %% 24)

plot(density(train_sample$hour[train_sample$isFraud == "no"], na.rm = TRUE),
     main = "Figure 2: Transaction Hour: Fraud vs Legit",
     xlab = "Hour of Day (0-23)", col = "blue", lwd = 2)

lines(density(train_sample$hour[train_sample$isFraud == "yes"], na.rm = TRUE),
      col = "red", lwd = 2)

legend("topright", legend = c("Legit", "Fraud"), col = c("blue", "red"), lwd = 2)
```

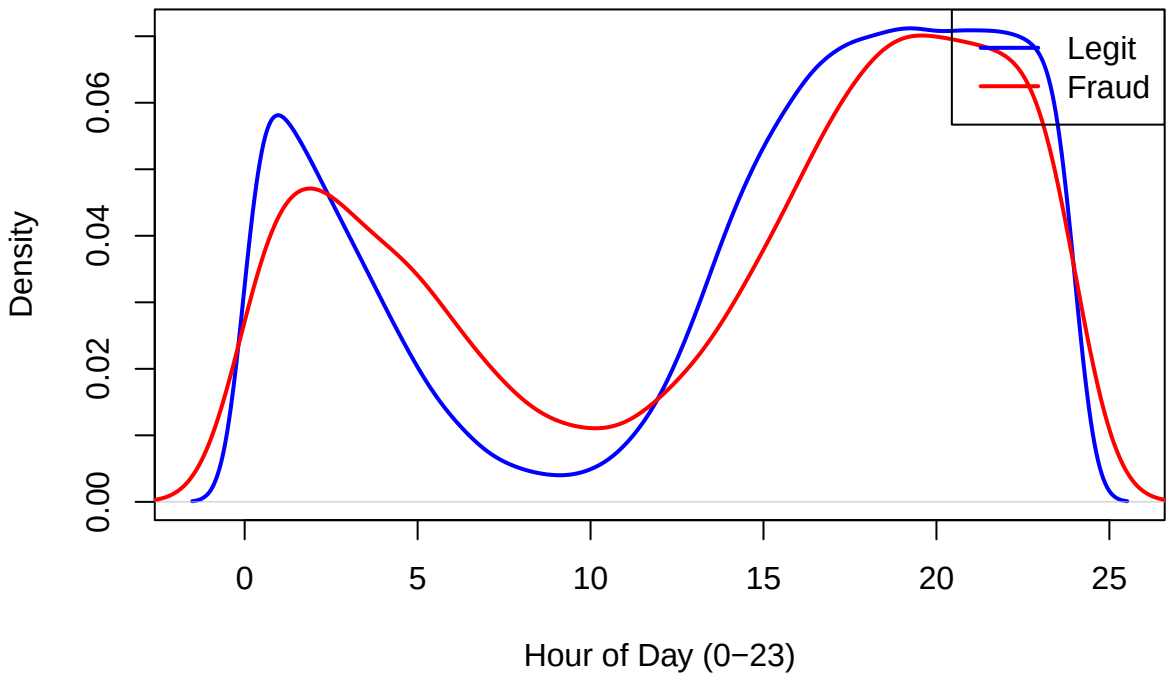


Figure 2 displays the density of transaction hour, derived from the TransactionDT timestamp, separated by fraud status. Both legitimate and fraudulent transactions share a similar overall pattern, with activity peaking in the evening hours around hour 19-20 and dipping in the mid-morning around hour 10. However, legitimate transactions show a more pronounced early-morning peak near hour 1-2, while fraudulent transactions are more evenly distributed throughout the day with a flatter trough during off-peak hours. This suggests that fraudulent activity is relatively more likely during daytime hours when legitimate activity is lower, which may reflect fraudsters operating across different time zones

```
train_sample |>
  count(isFraud) |>
  mutate(prop = n / sum(n)) |>
  kable(caption = "Target Variable Distribution")
```

Table 5: Target Variable Distribution

isFraud	n	prop
no	455902	0.9650088
yes	16531	0.0349912

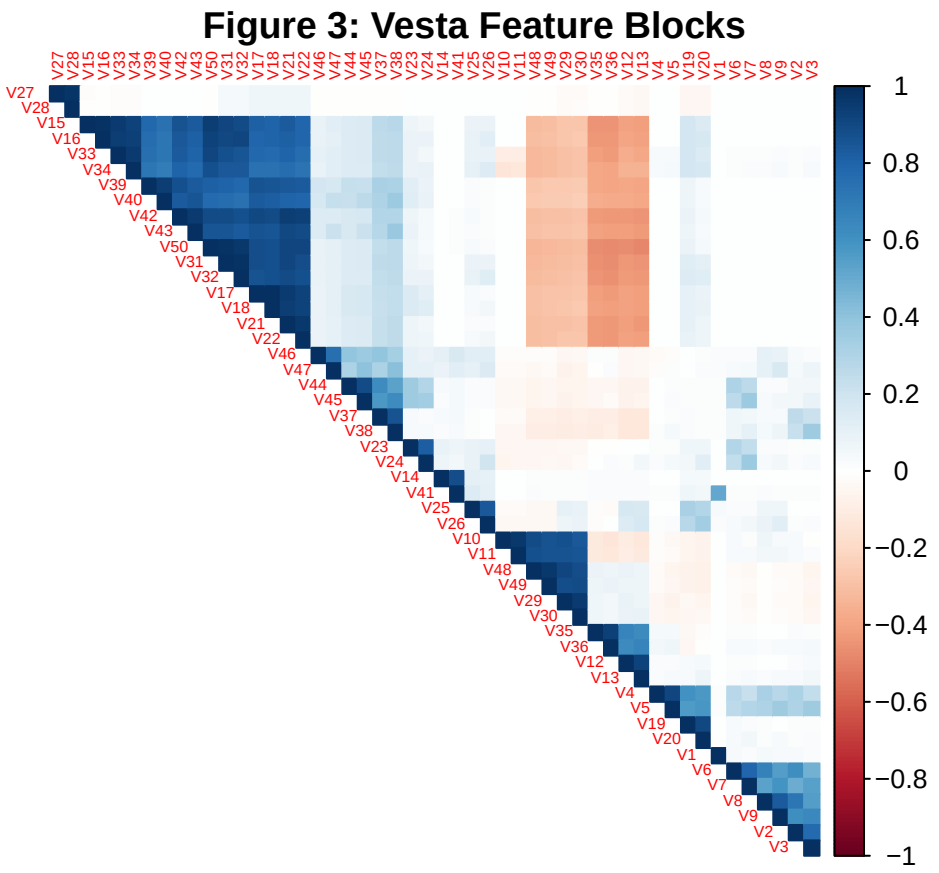
Table 5 displays the distribution of the target variable isFraud in the training sample. Of the 472,433 total transactions, 455,902 (96.5%) are legitimate and only 16,531 (3.5%) are fraudulent. This confirms the assumption of class imbalance, with fraudulent transactions representing a small minority of all observations.

```
v_numeric <- train_sample |>
  select(starts_with("V")) |>
  select(1:50) |>
  select(where(is.numeric)) |>
```

```
  select(where(~sd(.x, na.rm = TRUE) > 0))

# 2. Compute correlation
cor_v <- cor(v_numeric, use = "pairwise.complete.obs")
cor_v[is.na(cor_v)] <- 0

corrplot(cor_v,
         method = "color",
         type = "upper",
         order = "hclust",
         tl.cex = 0.5,
         title = "Figure 3: Vesta Feature Blocks",
         mar = c(0,0,1,0))
```



The full dataset contains 339 Vesta features in total, making it impractical to visualize all at once. Figure 3 displays a correlation heatmap of the first 50 Vesta engineered features (V1–V50), clustered by similarity as a representative sample. Several large clusters of strongly positively correlated features are visible in the upper left portion of the matrix, suggesting these features may be partially redundant.The high multicollinearity within these blocks suggests that tree-based models, which are robust to correlated features, may be a good fit for this dataset.

Model Building Process

V-fold Cross-Validation

The training data is further split into 5 folds stratified on the outcome, isFraud, for cross-validation.

```
fraud_folds <- vfold_cv(train_sample, v = 5, strata = isFraud)
```

Building Our Recipes

Create a base recipe that is shared across all models. First, the transaction hour is extracted from the raw TransactionDT timestamp as a new feature, after which TransactionDT is dropped. Any zero-variance predictors, which carry no discriminatory information, are removed. To handle categorical variables robustly, unseen factor levels that may appear in the test set are accommodated via step_novel, rare factor levels appearing in fewer than 2% of training observations are collapsed into a single "other_rare" category, and remaining missing factor levels are encoded as an explicit "Missing" category rather than being dropped. TransactionAmt, dist1, and dist2 are log-transformed to reduce right skew, and all remaining missing numeric values are imputed using the column median.

```
base_recipe <- recipe(isFraud ~ ., data = train_sample) |>
  step_mutate(hour = (TransactionDT / 3600) %% 24) |>
  step_mutate(across(where(is.logical), as.factor)) |>
  step_rm(any_of(c("TransactionDT"))) |>
  step_zv(all_predictors()) |>
  step_novel(all_nominal_predictors()) |>
  step_other(all_nominal_predictors(), threshold = 0.02, other = "other_rare") |>
  step_unknown(all_nominal_predictors(), new_level = "Missing") |>
  step_log(any_of(c("TransactionAmt", "dist1", "dist2")), offset = 1) |>
  step_impute_median(all_numeric_predictors())
```

Use the base to create two model specific recipes. For XGBoost, nominal predictors are encoded using step_dummy, as it requires numeric inputs.

```
# XGBoost
boost_recipe <- base_recipe |>
  step_dummy(all_nominal_predictors())
```

For logistic regression and elastic net: near-zero variance predictors are removed, nominal predictors are dummy encoded, any resulting zero-variance columns are dropped, and all numeric predictors are standardized to have mean zero and unit variance. Standardization is necessary for logistic regression and elastic net as these models are sensitive to the scale of input features, unlike tree-based methods.

```
# Logistic
logistic_recipe <- base_recipe |>
  step_mutate(across(where(is.logical), as.factor)) |>
  step_nzv(all_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_normalize(all_numeric_predictors())
```

Model Specs

Four model specifications are defined. Each specification declares the model type, the computational engine used to fit it, and which hyperparameters will be tuned during cross-validation versus fixed in advance.

Logistic regression is fit with no tuning, serving as a baseline. Elastic net extends logistic regression with two tuned regularization parameters that control the strength and type of penalty applied to the coefficients. Random forest is fit with 200 trees, with the number of randomly sampled predictors per split and the minimum node size left to be tuned. XGBoost has the most flexibility, with four tuned hyperparameters controlling the number of trees, tree depth, predictor sampling, and learning rate. Hyperparameter tuning for all models is performed using the 5-fold stratified cross-validation folds, with ROC AUC as the tuning metric.

```
log_spec <- logistic_reg() |>
  set_engine("glm") |>
  set_mode("classification")

en_spec <- logistic_reg(penalty = tune(),
                        mixture = tune()) |>
  set_engine("glmnet") |>
  set_mode("classification")

rf_spec <- rand_forest(mtry = tune(),
                       trees = 200,
                       min_n = tune()) |>
  set_engine("ranger") |>
  set_mode("classification")

boost_spec <- boost_tree(mtry = tune(),
                         trees = tune(),
                         tree_depth = tune(),
                         learn_rate = tune()) |>
  set_engine("xgboost") |>
  set_mode("classification")
```

Tuning grids are defined for each model that requires hyperparameter optimization.

For elastic net, a regular grid of 10 evenly spaced values is created for both penalty and mixture, ranging from pure ridge at 0 to pure lasso at 1.

For random forest and XGBoost, space-filling designs based on Latin hypercube sampling are used, with mtry ranging from 2 to 15. Given the large number of features in this dataset, a low upper bound for mtry is intentional. Allowing too many predictors to be considered at each split in a high-dimensional dataset risks individual trees becoming too similar to one another.

```
en_grid <- grid_regular(
  penalty(),
  mixture(range = c(0, 1)),
  levels = 10
)

rf_grid <- grid_space_filling(
  mtry(range = c(2, 15)),
  min_n(range = c(5, 30)),
  size = 10
```

```
)

boost_grid <- grid_space_filling(
  trees(range = c(100, 1000)),
  tree_depth(range = c(3, 10)),
  learn_rate(range = c(-3, -1)),
  mtry(range = c(2, 15)),
  size = 30
)
```

Workflows

Lastly, each recipe is paired with its corresponding model specification into a single workflow object, bundling together all the steps needed to train the models.

```
rf_wflow <- workflow() |>
  add_recipe(base_recipe) |>
  add_model(rf_spec)

boost_wflow <- workflow() |>
  add_recipe(boost_recipe) |>
  add_model(boost_spec)

log_wflow <- workflow() |>
  add_recipe(logistic_recipe) |>
  add_model(log_spec)

en_wflow <- workflow() |>
  add_recipe(logistic_recipe) |>
  add_model(en_spec)
```

Running Models

At this stage, each model is trained using its corresponding workflow and tuning grid across the 5-fold cross-validation folds. Logistic regression, which has no hyperparameters to tune, is fit using fit_resamples() and evaluated on ROC AUC and PR AUC across all five folds. The remaining three models, elastic net, random forest, and XGBoost, are tuned using tune_grid(), which fits each candidate hyperparameter combination across all five folds and selects the best performing configuration.

To avoid rerunning the tuning process, all results are saved as RDS files immediately after training and loaded back.

```
log_res <- fit_resamples(
  log_wflow,
  resamples = fraud_folds,
  metrics = metric_set(roc_auc, pr_auc),
  control = control_resamples(save_pred = TRUE)
)
saveRDS(log_res, "log_res.rds")

en_res <- tune_grid(
```

```
  en_wflow,
  resamples = fraud_folds,
  grid = en_grid,
  metrics = metric_set(roc_auc, pr_auc),
  control = control_grid(save_pred = TRUE)
)
saveRDS(en_res, "en_res.rds")

rf_res <- tune_grid(
    rf_wflow,
    resamples = fraud_folds,
    grid = rf_grid,
    metrics = metric_set(roc_auc, pr_auc),
    control = control_grid(
      save_pred = FALSE,
      parallel_over = "everything"
    ))

saveRDS(rf_res, "rf_res.rds")

boost_res <- tune_grid(
  boost_wflow,
  resamples = fraud_folds,
  grid = boost_grid,
  metrics = metric_set(roc_auc, pr_auc),
  control = control_grid(save_pred = TRUE)
)

saveRDS(boost_res, "boost_res.rds")
```

Hyperparameter Visualization and Model Selection

Now that all models are completed, load in the saved results for each model. After tuning, the top performing hyperparameter combinations for each model are extracted using show_best(), which returns the highest scoring configurations ranked by cross-validated ROC AUC. Also, lets plot the tuned hyperparameters for each model to visualize the performance.

```
log_res <- readRDS("log_res.rds")
show_best(log_res, metric = "roc_auc")
```

```
## # A tibble: 1 x 6
##   .metric .estimator  mean     n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 roc_auc binary     0.833     5 0.00154 pre0_mod0_post0
```

```
en_res <- readRDS("en_res.rds")
show_best(en_res, metric = "roc_auc")
```

```
## # A tibble: 5 x 8
##         penalty mixture .metric .estimator  mean     n std_err .config
```

```
##           <dbl>   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 0.0000000001        1 roc_auc binary     0.833     5 0.00154 pre0_mod010_post0
## 2 0.00000000129       1 roc_auc binary     0.833     5 0.00154 pre0_mod020_post0
## 3 0.0000000167        1 roc_auc binary     0.833     5 0.00154 pre0_mod030_post0
## 4 0.000000215         1 roc_auc binary     0.833     5 0.00154 pre0_mod040_post0
## 5 0.00000278          1 roc_auc binary     0.833     5 0.00154 pre0_mod050_post0
```

```r
autoplot(en_res, metric = "roc_auc") +
  labs(title = "Figure 4: Elastic Net Tuning Results",
       x = "Regularization Penalty (log scale)",
       y = "ROC AUC",
       color = "Mixture (0=Ridge, 1=Lasso)") +
  scale_color_brewer(palette = "RdYlBu") +
  theme_minimal() +
  theme(legend.position = "bottom",
        legend.title = element_text(size = 9),
        legend.text = element_text(size = 7))
```

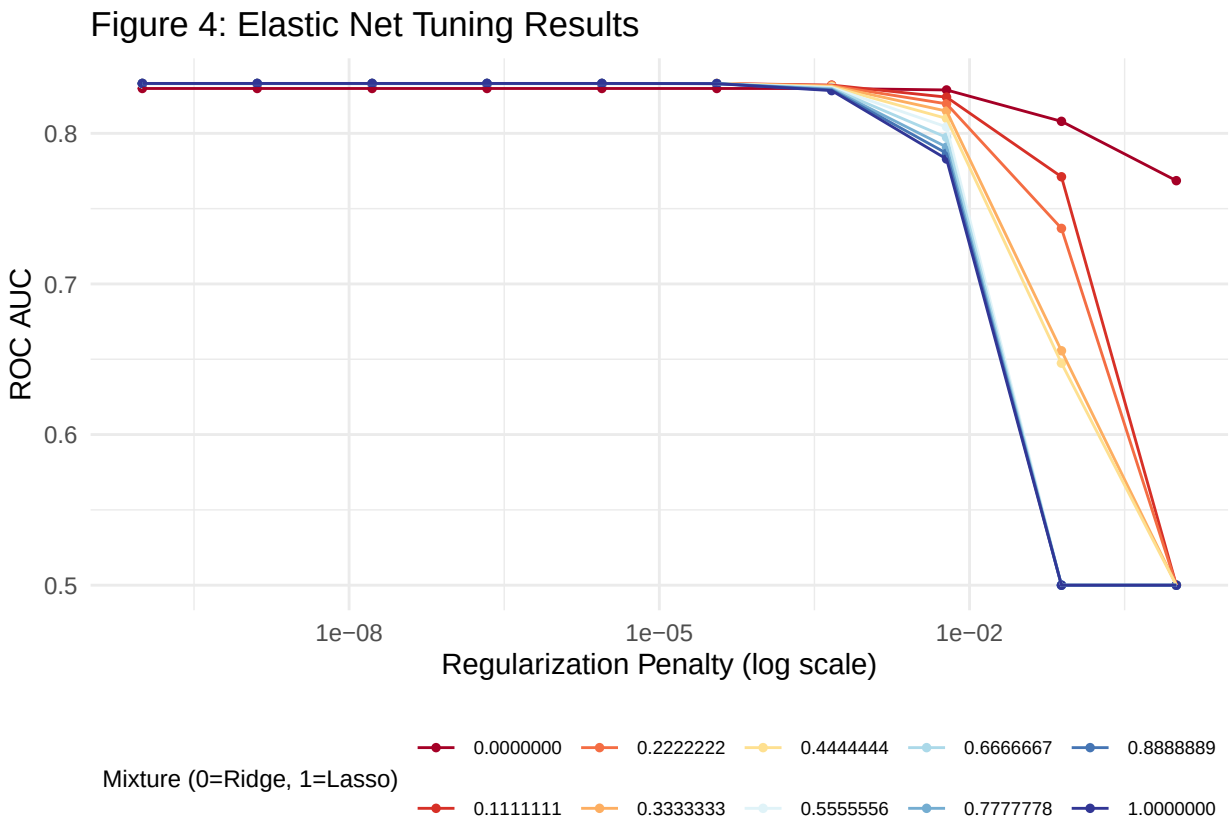


Figure 4 displays cross-validated ROC AUC across combinations of regularization penalty and mixture, which controls the proportion of lasso versus ridge penalty (color). All mixture values perform nearly identically at around 0.833 when the penalty is small, confirming that the linear model hits a hard ceiling regardless of regularization type.

```r
rf_res <- readRDS("rf_res_slim.rds")
show_best(rf_res, metric = "roc_auc")
```

```
## # A tibble: 5 x 8
```

```
##    mtry min_n .metric .estimator  mean     n  std_err .config
##   <int> <int> <chr>   <chr>      <dbl> <int>    <dbl> <chr>
## 1    13    10 roc_auc binary     0.929     5 0.00100  pre0_mod09_post0
## 2     9     7 roc_auc binary     0.927     5 0.000963 pre0_mod06_post0
## 3    15    21 roc_auc binary     0.927     5 0.00106  pre0_mod10_post0
## 4    10    18 roc_auc binary     0.924     5 0.000828 pre0_mod07_post0
## 5    12    30 roc_auc binary     0.924     5 0.00104  pre0_mod08_post0
```

```
autoplot(rf_res, metric = "roc_auc") +
  labs(title = "Figure 5: Random Forest Tuning Results",
       x = "Parameter Value", y = "ROC AUC") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

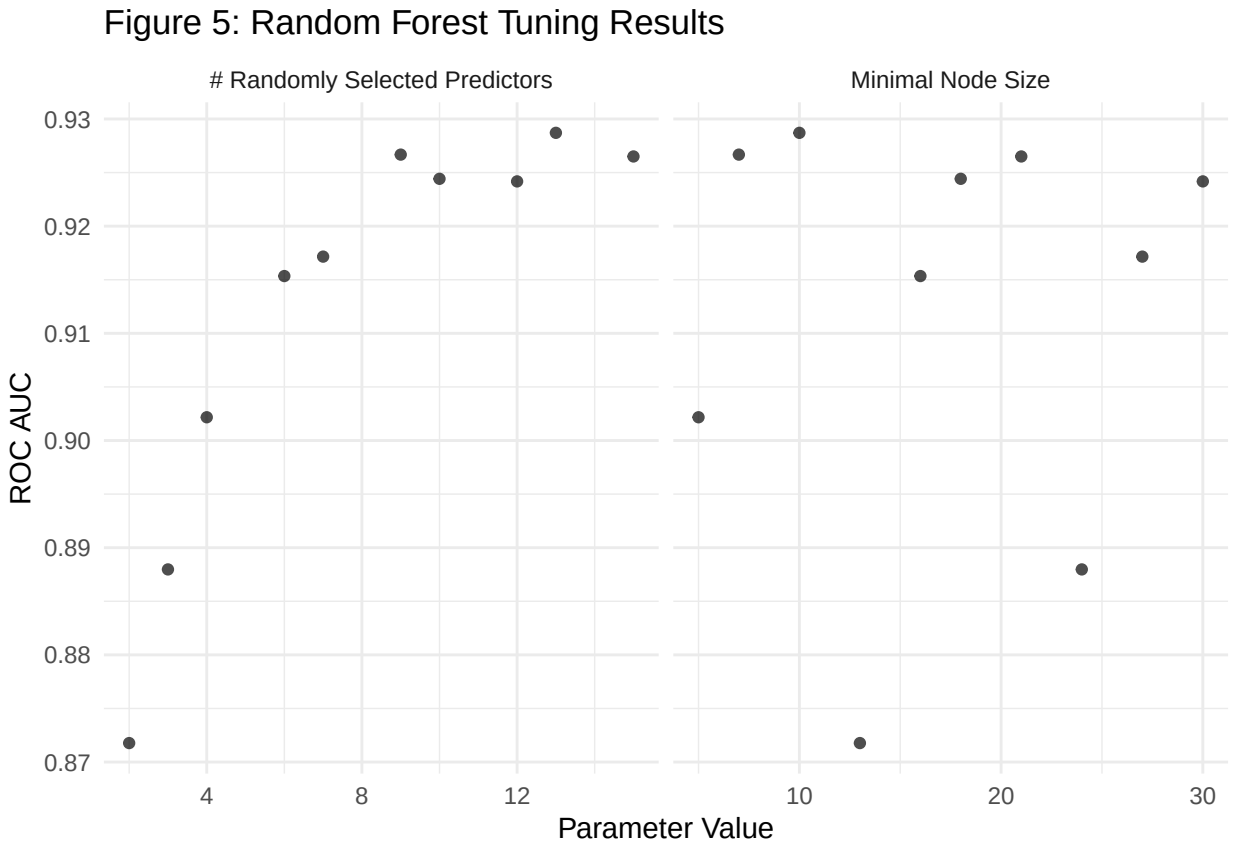


Figure 5 displays cross-validated ROC AUC across the two tuned hyperparameters for random forest: the number of randomly selected predictors (mtry) and the minimal node size (min_n). For mtry, performance improves steadily as the number of randomly selected predictors increases from 2, With the best clustering near 0.929 in the 9–13 range.

```
boost_res <- readRDS("boost_res_slim.rds")
show_best(boost_res, metric = "roc_auc")
```

```
## # A tibble: 5 x 10
##    mtry trees tree_depth learn_rate .metric .estimator  mean     n  std_err
##   <int> <int>      <int>      <dbl> <chr>   <chr>      <dbl> <int>    <dbl>
## 1    11   751          7     0.1    roc_auc binary     0.939     5 0.00121
```

```
## 2    12   565          10     0.0329 roc_auc binary     0.936     5 0.000801
## 3     4   782           9     0.0530 roc_auc binary     0.921     5 0.00127
## 4    10  1000           8     0.0174 roc_auc binary     0.921     5 0.00103
## 5     9   224           7     0.0853 roc_auc binary     0.913     5 0.00103
## # i 1 more variable: .config <chr>
```

```
autoplot(boost_res, metric = "roc_auc") +
  labs(title = "Figure 6: XGBoost Tuning Results",
       x = "Parameter Value",
       y = "ROC AUC") +
  theme_minimal() +
  theme(legend.position = "bottom")
```

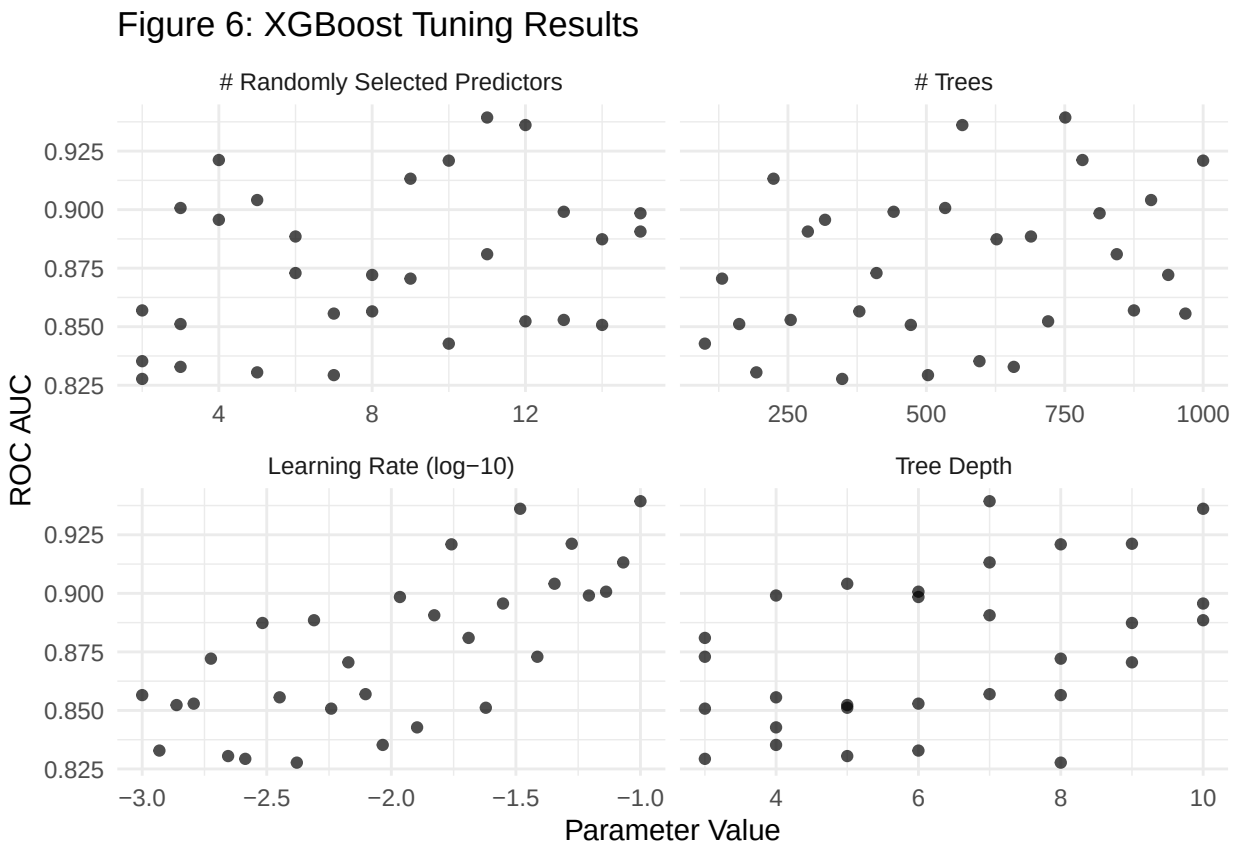


Figure 6 displays cross-validated ROC AUC across the four tuned hyperparameters for XGBoost. For the number of randomly selected predictors (mtry), performance improves as mtry increases toward the 11–13 range, echoing the pattern seen in random forest. For the number of trees, there is no strong directional trend, suggesting that the learning rate and tree depth play a more decisive role in determining final performance. Learning rate shows the clearest trend of all four parameters: configurations with lower learning rates (around -3.0 on the log scale) tend to perform poorly, while the best scores cluster toward the higher end of the range near -1.0 to -1.5, indicating that a more aggressive learning rate benefits this dataset. Tree depth also shows a positive trend, with deeper trees generally achieving higher ROC AUC. The best overall configurations tend to combine a moderate-to-high mtry, a higher learning rate, and greater tree depth.

We select the single best hyperparameter combination for random forest and XGBoost which will be used to finalize their respective workflows before fitting on the full training data.

```
rf_res <- readRDS("rf_res.rds")
best_rf <- select_best(rf_res, metric = "roc_auc")
```

```
boost_res <- readRDS("boost_res.rds")
best_boost <- select_best(boost_res, metric = "roc_auc")
```

Finalize the workflows.

```
final_rf_wflow <- finalize_workflow(rf_wflow, best_rf)
final_boost_wflow <- finalize_workflow(boost_wflow, best_boost)

final_rf_fit <- fit(final_rf_wflow, data = train_sample)
final_boost_fit <- fit(final_boost_wflow, data = train_sample)
```

Read in the saved final random forest and XGBoost fitted data.

```
final_rf_fit <- readRDS("final_rf_fit.rds")
final_boost_fit <- readRDS("final_boost_fit.rds")
```

The final models are used to make predictions on the held-out test set, and their performance is evaluated using ROC AUC and PR AUC to see how well each model detects fraud on data it has never seen before. Like when training the models, the results are saved and loaded in.

```
rf_preds <- augment(final_rf_fit, new_data = test_sample)
boost_preds <- augment(final_boost_fit, new_data = test_sample)
```

```
# load predictions
rf_preds <- readRDS("rf_preds.rds")
boost_preds <- readRDS("boost_preds.rds")
```

Interpreting Results (Random Forest & XGBoost)

```
rf_preds    |> roc_auc(truth = isFraud, .pred_no)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.933
```

```
boost_preds |> roc_auc(truth = isFraud, .pred_no)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.942
```

XGBoost achieves a test ROC AUC of 0.942, slightly outperforming random forest which achieves 0.933. Both results are consistent with and marginally higher than their respective cross-validated scores of 0.939 and 0.929, suggesting that neither model overfit during tuning and that both generalize well to unseen data.

```
rf_preds    |> pr_auc(truth = isFraud, .pred_yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 pr_auc  binary         0.889
```

```
boost_preds |> pr_auc(truth = isFraud, .pred_yes)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 pr_auc  binary         0.888
```

Evaluated on PR AUC, random forest and XGBoost perform nearly identically at 0.889 and 0.888 respectively, with random forest marginally edging out XGBoost by a negligible margin. Given the severe class imbalance in the dataset, PR AUC is a particularly informative metric as it focuses specifically on model performance among predicted fraud cases. The near identical PR AUC scores suggest that both models are equally capable of identifying fraudulent transactions, despite XGBoost holding a slight edge in ROC AUC. Overall, both models demonstrate strong performance across both metrics, with XGBoost being the preferred model based on its higher ROC AUC.

```
# XGBoost
final_boost_fit |>
  extract_fit_parsnip() |>
  vip(num_features = 20) +
  labs(title = "Figure 7: XGBoost Feature Importance (Top 20)",
       x = "Variable",
       y = "Importance") +
  theme_minimal() +
  theme(panel.grid.major.y = element_blank())
```

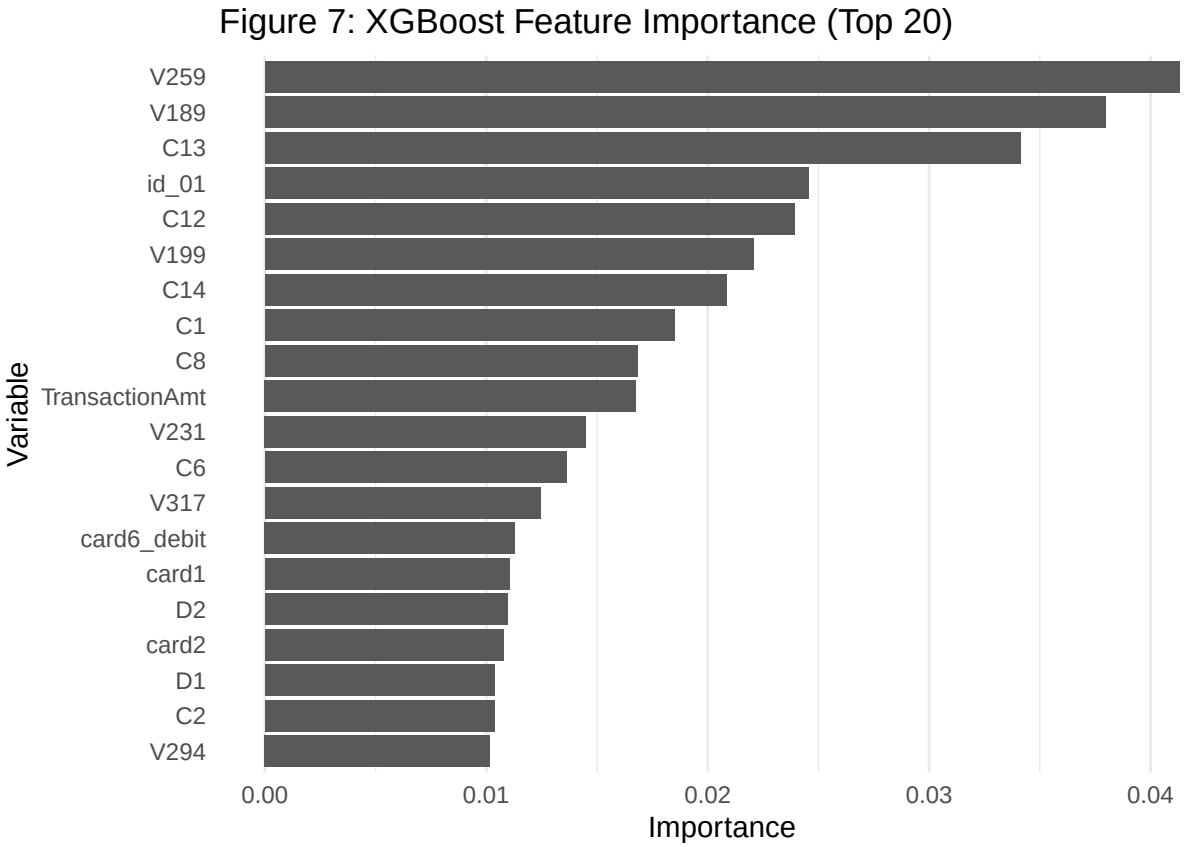


Figure 7 displays the top 20 most important features from the final XGBoost model. The two Vesta engineered features V259 and V189 are the most influential predictors by a clear margin, followed by C13, which represents the count of addresses associated with the payment card. The prominence of C13 and several other C features suggests that card variables are strong indicators of fraud, capturing behavioral patterns such as a card being used across many different addresses or accounts, which is a common signal of stolen credentials. The identity feature id_01 and TransactionAmt also rank highly. The D features D1 and D2, which represent time-based delta features, also appear, suggesting that the timing patterns between transactions are informative.

Confusion Matrix

A confusion matrix is a helpful tool to see where the model is making mistakes.

```
rf_preds |> conf_mat(truth = isFraud, estimate = .pred_class)
```

```
##           Truth
## Prediction     no    yes
##        no  113865   2466
##        yes    110   1666
```

```
boost_preds |> conf_mat(truth = isFraud, estimate = .pred_class)
```

```
##           Truth
## Prediction     no    yes
##        no  113831   2026
##        yes    144   2106
```

XGBoost catches significantly more fraud (2,106 vs 1,666 true positives) but also flags slightly more legitimate transactions as fraud (144 vs 110 false positives). Random forest rarely flags legitimate transactions, but also misses more fraud.

Visualize the ROC AUC Curves

It can be helpful to visualize the ROC AUC curve, especially since both models were so close in ROC AUC and PR AUC.

```
bind_rows(
  rf_preds    |> roc_curve(truth = isFraud, .pred_no) |> mutate(model = "Random Forest"),
  boost_preds |> roc_curve(truth = isFraud, .pred_no) |> mutate(model = "XGBoost")
) |>
  ggplot(aes(x = 1 - specificity, y = sensitivity, color = model)) +
  geom_line(linewidth = 1) +
  geom_abline(lty = 2) +
  labs(title = "Figure 8: ROC Curve", x = "1 - Specificity", y = "Sensitivity") +
  theme_minimal()
```

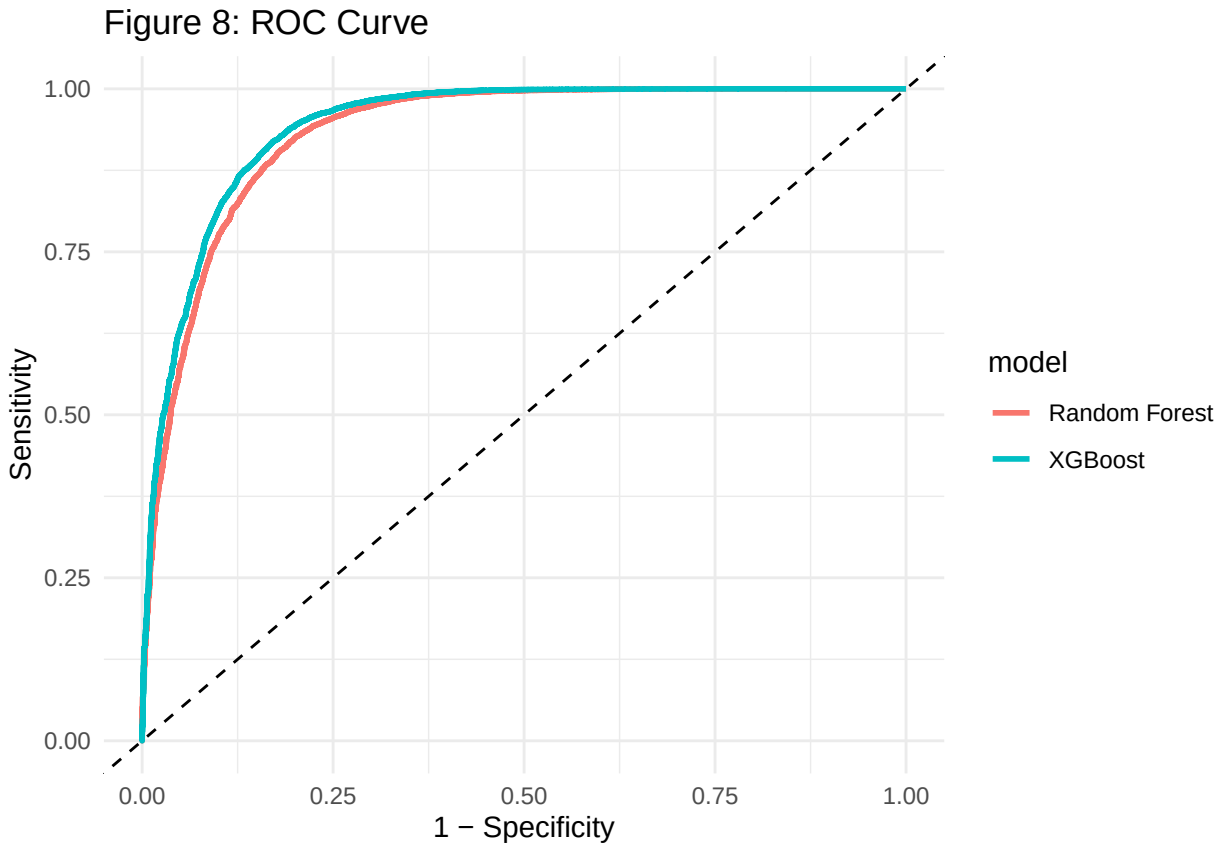


Figure 8 displays the ROC curves for both final models on the held-out test set. Both curves bow substantially toward the upper left corner, indicating strong performance well above the random baseline represented by the dashed diagonal line. XGBoost's curve sits consistently above random forest's throughout. The most notable separation between the two models occurs on the left side of the plot, where XGBoost is more sensitivity for the same false positive rate. This also confirms that its higher ROC AUC of 0.942 reflects a genuine and consistent advantage.

Conclusion

Financial fraud represents one of the most pressing challenges facing digital commerce today.The ability to identify fraudulent transactions in real time has become crucial for financial institutions as e-commerce grows and fraudsters become more skilled. This project tackled that problem using the IEEE-CIS Fraud Detection dataset, a large-scale real-world dataset of anonymized transactions released by Vesta Corporation. Four classification models were trained and evaluated: logistic regression, elastic net, random forest, and XGBoost. Given the severe class imbalance in the data, where fraudulent transactions represent only 3.5% of all observations, models were assessed using ROC AUC and PR AUC rather than accuracy, as accuracy alone would be misleading.

The results told a clear and consistent story. Logistic regression and elastic net both plateaued at a cross-validated ROC AUC of 0.833, and no amount of regularization could push them further. Random forest and XGBoost broke through that ceiling, achieving cross-validated ROC AUC scores of 0.929 and 0.939 respectively, both of which held up on the held-out test set at 0.933 and 0.942. On PR AUC, which is particularly meaningful under class imbalance, both models performed nearly identically at 0.889 and 0.888. XGBoost was selected as the best overall model, and its feature importance analysis revealed that Vesta's engineered V-features dominated the top predictors, followed by behavioral card variables.

Several directions remain open for future work. The most immediate opportunity is scale: because the competition's test set was unlabeled, the available training data had to serve as the training and test data with a 80/20 split. Training on the complete labeled dataset, perhaps through a different validation strategy, could yield meaningfully different results. Furthermore, more rigorous feature engineering, such as creating interaction terms between transaction amounts and specific product codes, could further improve the model's precision. Finally, exploring deep learning architectures could help the model learn more effectively from the sparse fraud cases, potentially pushing performance beyond the current tree-based limits.